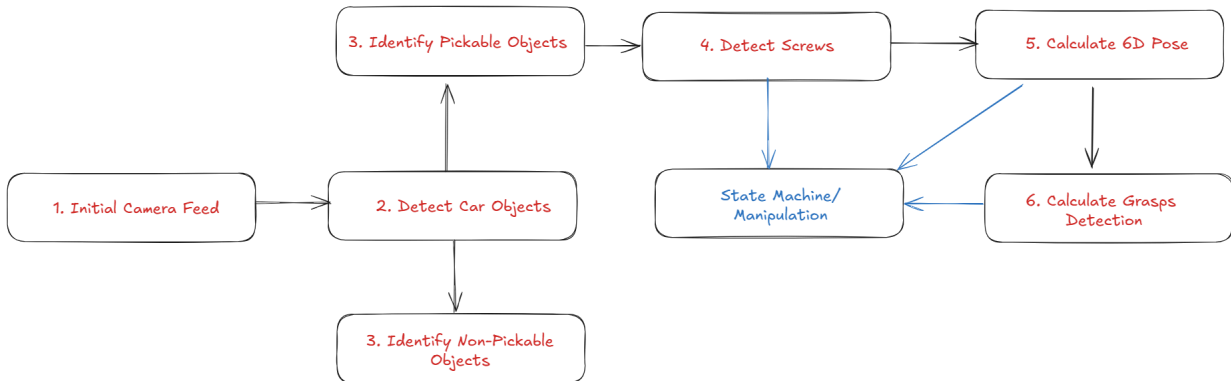


Plan For Perception Team



1. Initial Camera Feed:

- Kinova Robot Vision (https://github.com/PickNikRobotics/ros2_kortex_vision)
- Realsense Camera (<https://github.com/IntelRealSense/realsense-ros>)

2. Detect Car Objects (Bounding Boxes)

- Use YOLOv8 (<https://github.com/ultralytics/ultralytics>)
- Publish the bounding boxes as a `/detections` topic for ROS support

3. Identify Pickable Objects and Non-Pickable Objects

- Use ROS to subscribe to the `/detections` topic from YOLO node, classifying objects as "pickable" or "non-pickable" based on their class ID, and publishing them to two separate topics.
- Pickable Objects: Motor, Unit
- Non Pickable By Robot Objects: Speaker, Wire_plug, Enclosure

4. Detect Screws (Bounding Boxes)

- Use YOLOv8 (<https://github.com/ultralytics/ultralytics>)
- Possibility of using other datasets such as
 - <https://universe.roboflow.com/capjamesg/counting-screws/dataset/8>
 - https://universe.roboflow.com/wojciech-machnicki-kkysb/screw_nut-xyamk/images/1oC9mZBsOc0ThEKiCken
 - Further Finetuning and Collection of dataset is required
- Publish the bounding boxes as a `/screw_detections` topic for ROS support

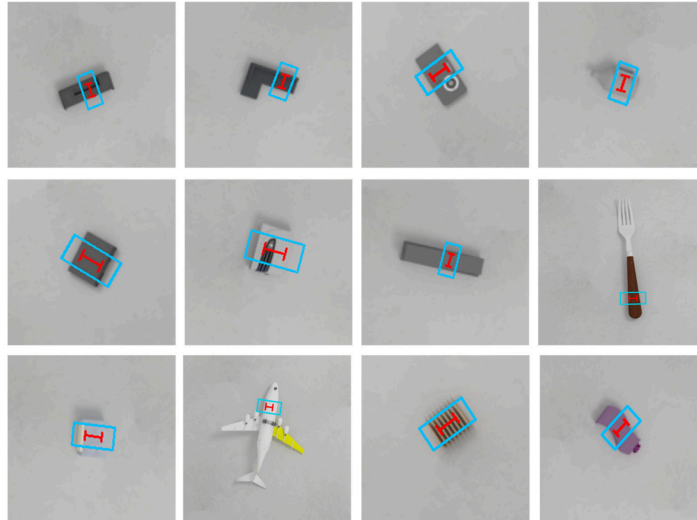
5. Pose Estimation (PCA)

- Crop the image for each of the bounding boxes for the detections
- Apply PCA to estimate 6D Pose of the objects
- Potential to explore other Pose Estimation Libraries

Plan For Perception Team

6. Grasp Detection:

a. 2D Grasps Detection:



Pros:

- Easy to annotate
- Easy to calculate and train a model

Cons:

- Works for only a fixed orientation i.e the camera of the robot needs to always be above an object directly looking down on it
- If an object has a large width, that can only be picked up from the side for example a bowl.

b. 6D Grasps Detection:

Pros:

- More accurate 6D Pose for grasping an object

Model Base Approach: Requiring a 3D model

Cons:

- Labeling on 3D custom objects is difficult
- Gathering 3D objects (using Iphone's Lidar + Camera for Photogrammetry), also optimizing the mesh to work on low-end devices
- Requires training time for training on a 3D dataset

Resources:

- <https://link.springer.com/article/10.1007/s00170-024-14372-3>

Model Free Approach: Using a high end gpu to create a virtual 3d object

Cons:

- Requires a high end gpu

Resources:

- <https://pku-epic.github.io/GraspNeRF/>

Plan For Perception Team

- <https://graspnet.net/anygrasp.html> (Requires a License)

Conclusion for Grasps Detection: Still requires more research and experimentation for the approach to be applied for the next milestone. Suggestion to go for **Rectangle 2D grasp** pose first and then for **6D Grasp Pose**.

7. State Machine/ Manipulation Team:

- The perception team will be responsible for sending the appropriate grasp poses and center poses of the object to the State Machine
- The State Machine will have a buffer of frames (i.e the perception team will be spamming with the /detections topic)
- The State Machine will be responsible for the average/randomly selecting one frame from the buffer and locking it to use for the manipulation team to grab the object. This is done to reduce the noise and fluctuations of the poses caused by the perception team.
- The State Machine requires to subscribe to the /object_pose topic